

Matmul 之外：16 个 GPU Benchmark 的 Jac 功能分析

每项选一种常用算法，解释计算、布局、OSP 与 GPU 后端要求

16

公认 GPU 任务 · 常用算法 · 具体功能与运行协议

BabelStream · Rodinia · NVIDIA GPU libraries / samples · Gunrock

三页 / 任务：算法与证据 → 布局与 Jac → 评测定义与微型例子

代码能力快照：b24603463 | 本报告独立于 40 种 matmul PDF

交付为算法/功能分析与自定义 benchmark 协议；没有运行 Jac/GPU 性能测试。

文中的 16 个微型数值例子已由 Python CPU 模型复核；不等于 Jac/GPU benchmark 已实现。

为何选这些任务：用来源支撑 **GPU** 适用性

四类可靠的选题证据

- 正式 benchmark / suite : BabelStream 的带宽任务, Rodinia 的 K-means、BFS、Needleman-Wunsch。
- 成熟 GPU 库 : CUB 的 reduce/scan/radix sort , cuSPARSE SpMV、cuFFT、cuDNN softmax。
- 官方 GPU 算法示例 : histogram、separable convolution、Jacobi、N-body、Black-Scholes。
- GPU 图框架 : Gunrock 的 frontier 与 PageRank ; 用实际 GPU 计算模式, 而非名字判断。

这份报告如何使用这些证据

- 为每个任务固定一种常用算法、输入域、布局、规模和正确性规则, 形成可实现的自定义基准定义。
- 官方 samples 是适用性证据, 官方说明它们并非规范性能 benchmark ; 这里不宣称官方成绩。
- GPU 适合这些任务的可扩展并行结构 ; 实际 speedup 仍取决于规模、驻留、数据分布与 GPU。
- 除数学微例复核外, 本轮没有执行这些 Jac/GPU benchmark ; 功能状态来自源码审阅。

依据 : BabelStream、Rodinia、NVIDIA CUDA Samples 的用途说明。功能/调度设计与规模选择为本报告工程分析。

16 个任务与选定算法：本报告不含新 matmul 变体

每任务三页。大规模默认 float64 / int64，FFT 为 re/im 两个 float64 列；扩展 FP32 要单独记录 dtype。

Benchmark	本报告选定的常用算法	开始页
01 triad	每元素独立 fused Triad	7
02 reduction	分层树形并行归约	10
03 scan	Blelloch work-efficient scan + 分层块偏移	13
04 histogram	块私有直方图 + 全局合并	16
05 radix_sort	稳定 LSD radix sort，4 bit 一轮	19
06 spmv	CSR 行并行 SpMV（可选 warp-per-row）	22
07 jacobi	5-point Jacobi，双缓冲逐轮更新	25
08 convolution	可分离 Gaussian：水平 + 垂直两遍	28
09 fft	radix-2 Cooley-Tukey DIT，bit-reversal 输入	31
10 nbody	分块 all-pairs softened gravitational force	34
11 kmeans	Lloyd：最近中心分配 + 分组重算中心	37
12 bfs	level-synchronous frontier BFS（push）	40
13 pagerank	pull power iteration + dangling mass correction	43
14 alignment	Needleman-Wunsch，anti-diagonal wavefront	46
15 black_scholes	欧洲 call/put 闭式公式 + normal CDF	49
16 softmax	stable max-subtract → exp/sum → normalize	52

当前 Jac GPU 能力：哪些可以复用

已支持的组成部分

- int64 或 float64 标量，bool 可混合；固定 plain obj 的标量叶子 SoA 展开。
- here 标量读取、self 私有状态更新；+/-/*、整数 %、比较/布尔逻辑与 if。
- 单一空属性 typed edge 的 tail visit；链式/实验 CSR FIFO，允许分支与重复 arrival。
- 一线程完成一个独立 walker；只读共享图、私有结果/report、batch 打包与状态回写。

所选常用算法还需要什么

- 通用设备 buffer/view、一般循环、动态 gather/scatter、int 索引与 float 数据混合。
- 全局/分组归约、scan、共享输出原子、frontier 与阶段依赖；非私有状态的写入契约。
- 数学函数、除法、bitwise/shift、复数数组；不是所有已有 CPU Jac 调用都能发射 PTX。
- 标准 buffer 路径均未在本任务跑通；少量表达式/单轮 CSR 结构匹配只标为可复用，未验证。

能力依据：ptx_walker.jac::chain_expr / chain_update / project_fields；ptx_csr.jac；runtime/gpu.jac。

功能标签：与 **matmul** 报告保持对应，新增 **M**

标签表示算法涉及的能力域；不表示当前已支持。最低要求与条件优化在各任务单页明确区分。

D 设备数据

typed buffer/view、shape/stride、共享与所有权、输入驻留、workspace。

I 控制与寻址

for/while、局部变量、整数地址、gather/scatter、边界、静态 helper。

P 调度与输出

一任务/线程、独占输出、共享状态、CAS/atomic、隐式任务坐标。

R 集合与阶段

sum/max、scan、分组归约/compaction、wavefront、多 kernel 依赖。

G **OSP** 图语义

node/edge handle、query/receiver、edge payload、异构 schema、隐式关系。

H 协作与内存

warp/block、shared memory、barrier、寄存器 tile、布局选择。

M 数学与位运算

除法、sqrt/exp/log/erfc、复数拆解、shift/bitmask/bit reversal。

L **GPU** 互操作

tensor ABI、host FFI、GPU 库/compiled wrapper、context/stream 与完成。

N 数值与语义

dtype、求和顺序、精确整数、稳定排序、tie、误差/确定性。

统一 **benchmark** 协议：性能与正确性分别定义

输入、**dtype** 与计算范围

- float64/int64 为分析基线，便于对应现有 GPU scalar 类型；radix key 逻辑 unsigned32 由 int64 承载。
- 随机 seed 固定为 20261002；报告输入规模、分布、算法参数、拓扑和 checksum。
- 每次迭代按定义重置状态；初始化/格式转换/构图/plan 单独列出，不能悄悄算到 CPU。
- 若增加 FP32/complex64/uint8 布局，需扩展 dtype ABI，并只做同精度性能对照。

建议测量与验证（尚未执行）

- 5 次 warmup，20 次有效重复；GPU 完成后测 median/p90，记录设备、驱动、编译/库版本。
- 分别报告 resident kernel、完整算法阶段和端到端时间；异步调用用 GPU events/同步确认完成。
- 整数与图 distance 精确比较；float64 微例可用 $1e-12$ ，主规模按算法误差另定容差。
- 库对照可有不同内部算法，但输入域/结果语义须相同；本报告给协议，不给未测的 speedup。

规模是建议 sweep，不是已跑结果；workspace/可用显存和所有阶段成本需由后续实现实际记录。

01 triad：向量带宽：STREAM-style Triad

选定算法：每元素独立 fused Triad

任务定义与算法步骤

$A[i] = B[i] + \alpha * C[i]$ ； α 固定，B/C 只读，A 为新输出。

- N 个输出彼此独立，一线程/工作项负责一个或若干连续 i。
- 算法 $O(N)$ ，每元素一次乘加；基线使用 float64，输入和输出三个连续数组。
- 选取 BabelStream 的常见 Triad 计算作为自定义微基准；本报告不提交正式 STREAM 成绩。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
parallel for i in [0,N):  
    A[i] = B[i] + alpha*C[i]
```

为什么是公认 GPU 任务

- GPU 适用性证据：BabelStream 提供 CUDA 等多种 GPU 实现，用于考察内存带宽。
- 足够大的 N 提供大量独立工作；lane 沿 i 连续，输入读取和输出写入容易合并。
- 用于把算术与内存布局的能力分开；有效字节数为 $24N$ （float64），不含上传。

适用规模 / 输入条件

小 N 或每次必须上传/回传时，启动和 PCIe 成本可能占主导；不预设 speedup。

01 triad：数据布局、OSP 与 Jac 要求

选择算法保持一致；最低可执行要求与高性能扩展分开。各项功能在后文词典解释。

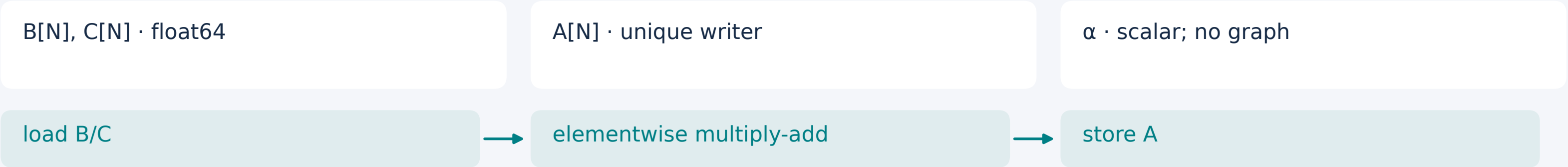
D 设备数组 / view

I 索引 / 控制流

P 任务 / 输出 / 原子

N 数值语义

物理表示（共享输入 / 独占或显式共享输出）



OSP 表达：逻辑上可用一个元素任务/walker；任务坐标由 lane 产生即可，不必构建 N 个 host 节点。

最低 GPU 运行：Jac 必须提供

- D/I：连续设备数组、整数下标与 load/store；float64 标量乘加已有。
- P：一工作项独占 A[i]；坐标生成与有限计算型 entry / launch。
- 运行时：共享只读 B/C、独立 A 与完成后回传；无跨线程归约。

高性能扩展 / 复用情况

- 相邻 lane 沿 i；向量化 load/store 与 grid-stride 循环可提升吞吐。
- 输入长期驻留、复用分配；记录实际与有效带宽，避免把缓存热数据当 DRAM 带宽。

当前状态

单点 float64 节点表达式与空 tail visit 结构匹配；标准 buffer 基准仍需 D/I/P，未运行。

当前 GPU 后端能力依据：ptx_walker.jac / ptx_csr.jac；映射与扩展建议未实机验证。

01 triad : benchmark 定义与正确性

以下为建议运行协议。微型例子由 Python CPU 数值模型复核；没有执行该 Jac/GPU benchmark。

输入规模、状态重置与指标

- $N=2^{20} / 2^{24} / 2^{26}$; $N=2^{24}$ 时三个 float64 数组共384 MiB。
- 固定 $\alpha=2$; 输入随机有限值 ; 基准每轮写新 A , 不修改 B/C。
- GPU 驻留 kernel 与端到端分别计时 ; 超出可用显存时减小N 并记录。

报告什么

有效 GB/s = $24N / \text{kernel_seconds} / 1e9$; 报告 dtype、N 和时间分位数。

计算 / 数值示意 (不是 Jac/GPU 运行结果)

```
B=[1,2], C=[3,4], alpha=2
A=[7,10]
```

CPU 参考与关键检查

CPU 标量 Triad 或 NumPy 数组运算 ; 整数微例精确 , 随机浮点按 abs+rel 容差。

- 尾部 N 不是 block 整数倍 ; 输入不变 ; 逐元素结果位置正确。
- 明确 FMA 融合是否允许 ; 同精度/同数据比较 , 计时不能混入初次JIT

GPU 对照 / 库路径

BabelStream CUDA Triad 或等价 elementwise GPU kernel ; 不把 Python CPU 调用当 GPU baseline。

02 reduction : 全数组求和

选定算法：分层树形并行归约

任务定义与算法步骤

输入 $x[N]$ ，输出一个 sum ；固定一棵分层归约结构。

- 先按块计算 $partial$ ；每块内部按树合并，再对 $partial$ 重复归约。
- 总工作 $O(N)$ ，阶段数与层次有关；不是一个线程沿 N 项串行累计。
- $float64$ 样例比较数值容差；树顺序与 CPU 左折叠通常不逐位相同。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
partial[b] = tree_sum(x[tile_b])
repeat until one result:
    partial = tree_sum_groups(partial)
```

为什么是公认 GPU 任务

- GPU 适用性证据：NVIDIA CUB 提供 device-wide reduction，并分为 thread/warp/block/device 层次。
- 各块读取不同连续 tile；归约是 GPU 数值和数据并行任务的基础算子。
- 输入大时可并行读取大量数据；尾部和部分结果应有定义。

适用规模 / 输入条件

只有一个最终输出；小数组或非常多微小归约可能受启动成本限制。

[REDUCE] NVIDIA CUB : device-wide reduction

02 reduction：数据布局、OSP 与 Jac 要求

选择算法保持一致；最低可执行要求与高性能扩展分开。各项功能在后文词典解释。

D 设备数组 / view

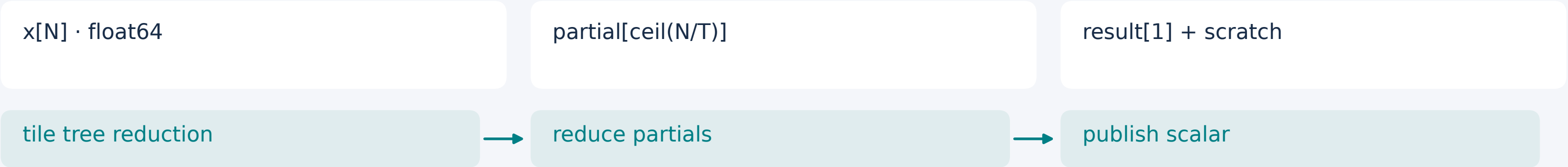
I 索引 / 控制流

P 任务 / 输出 / 原子

R 归约 / scan / 阶段

N 数值语义

物理表示（共享输入 / 独占或显式共享输出）



OSP 表达：叶子是输入项，逻辑树表示合并关系；物理树应由阶段/工作项表达，不要求 CPU 实体化所有树节点。

最低 GPU 运行：Jac 必须提供

- D/I/P：输入/partial buffer、边界掩码与互不覆盖的 partial 写入。
- R：树式合并、单位元 0、阶段完成顺序与最终标量输出。
- 最低可用逐层 kernel；不要求 device recursion 或全 grid 的块内barrier。

高性能扩展 / 复用情况

- H：warp shuffle / shared-memory block reduction，减少层次 kernel 与临时读写。
- L：接入 CUB；具体库 API/可重复性以安装版本为准。

当前状态

私有标量 += 已有；串行 chain sum 不等价于选定的分层并行归约，R/H 尚缺。

当前 GPU 后端能力依据：ptx_walker.jac / ptx_csr.jac；映射与扩展建议未实机验证。

02 reduction : benchmark 定义与正确性

以下为建议运行协议。微型例子由 Python CPU 数值模型复核；没有执行该 Jac/GPU benchmark。

输入规模、状态重置与指标

- $N=2^{20} / 2^{24} / 2^{26}$; float64 输入分别 8 / 128 / 512 MiB。
- 正负随机值、全零、恒定值与抵消序列；固定 tile / 归约分组。
- 计入所有 partial 阶段；额外 workspace 单列，不只测第一轮。

报告什么

kernel 总时间、元素/s；有效读取量可用 $8N$ 作输入下限，另报告 partial 流量。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
x=[1,2,3,4]
partial=[3,7] -> sum=10
```

CPU 参考与关键检查

小整数对精确整数 sum；浮点对 math.fsum / 高精度参考，并记录误差。

- 非二次幂 N 、尾块、全零、抵消与极小/极大量级组合。
- 重复运行稳定性与跨设备一致性分开验证；固定树不代表与 CPU 逐位相同。

GPU 对照 / 库路径

CUB DeviceReduce；库的内部树可能不同，比较同一数值契约与完整调用时间。

03 scan : 前缀和 / exclusive scan

选定算法 : Blelloch work-efficient scan + 分层块偏移

任务定义与算法步骤

$y[i] = \text{sum}(x[0:i])$; 整数输入 , $y[0]=0$, 不含 $x[i]$ 。

- 块内 upsweep 汇总 , 根设 0 , 再 downsweep 生成 exclusive 前缀。
- 大数组再 scan 块总和 , 把块偏移加到该块所有输出。
- 算法总体 $O(N)$; 它保留每个前缀 , 而不只返回最后一个总和。

计算 / 数值示意 (不是 Jac/GPU 运行结果)

```
local_y, block_sum = blelloch_scan(x_tiles)
block_offset = exclusive_scan(block_sum)
y[i] = local_y[i] + block_offset[block(i)]
```

为什么是公认 GPU 任务

- GPU 适用性证据 : GPU Gems 专章展示 CUDA scan ; CUB 也提供 device-wide scan。
- scan 广泛用于 compaction、scatter 定位、稀疏 offsets 和 frontier 生成。
- 规则输入/输出与块内树有大量并行工作 ; 选定 Blelloch 是常用教学/基础算法。

适用规模 / 输入条件

CUB 的高性能实现不必使用同一 Blelloch 分层树 ; 库路径是语义/ 性能对照。

03 scan：数据布局、OSP 与 Jac 要求

选择算法保持一致；最低可执行要求与高性能扩展分开。各项功能在后文词典解释。

D 设备数组 / view

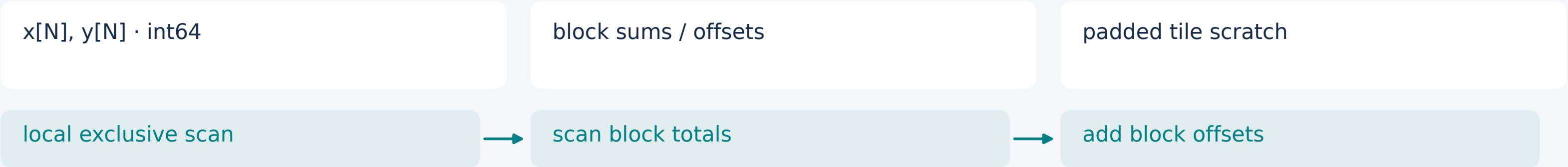
I 索引 / 控制流

P 任务 / 输出 / 原子

R 归约 / scan / 阶段

N 数值语义

物理表示（共享输入 / 独占或显式共享输出）



OSP 表达：逻辑依赖不是一次 visit 的总和；可以把数组 tile 定义为 scan 任务，无需一节点一前缀链。

最低 GPU 运行：Jac 必须提供

- D/I/P：有界数组、索引、各 tile 的输出所有权和加偏移 kernel。
- R：scan 的 exclusive 语义、块总和递归/阶段依赖；补零只用于算法尾部。
- N：int64 不溢出的输入范围；空输入/单元元素输出规则。

高性能扩展 / 复用情况

- H：块内 shared-memory tree 与 barrier；可用 shuffle 优化局部段。
- L：CUB scan；比较全部阶段和 workspace，不只计算局部前缀。

当前状态

没有一般数组/scan；按 chain report 串行输出前缀只是另一种算法与执行模型。

当前 GPU 后端能力依据：ptx_walker.jac / ptx_csr.jac；映射与扩展建议未实机验证。

03 scan : benchmark 定义与正确性

以下为建议运行协议。微型例子由 Python CPU 数值模型复核；没有执行该 Jac/GPU benchmark。

输入规模、状态重置与指标

- $N=2^{20} / 2^{24}$; int64 输入与输出在 $N=2^{24}$ 共 256 MiB , 另加块 workspace。
- 小非负整数；累加上界小于 int64 最大值；测试非二次幂与 $N=0/1$ 。
- 固定 exclusive scan ; 不要与 inclusive scan 成绩/答案混用。

报告什么

总元素/s、完整 scan 时间、临时空间；原始读写下限 $16N$ 字节。

计算 / 数值示意 (不是 Jac/GPU 运行结果)

```
x=[3,1,7,0]
exclusive y=[0,3,4,11]
```

CPU 参考与关键检查

CPU 精确整数 prefix ; 检查 $y[i+1]-y[i]=x[i]$, 总和与末端关系正确。

- N 跨越多个 tile , 块偏移不能丢 ; 尾部 padding 不应成为真实输出。
- 覆盖全零与长常量段 ; 验证 scan 输出可作为合法 scatter 位置。

GPU 对照 / 库路径

CUB DeviceScan ExclusiveSum ; 版本和整数 dtype 固定 , 调用完成后计读出。

04 histogram：分箱计数 / histogram

选定算法：块私有直方图 + 全局合并

任务定义与算法步骤

输入 $\text{bin_id}[N] \in [0, B)$ ，输出 $\text{count}[B]$ ；默认 $B=256$ 。

- 各块先对其输入 tile 计算 $\text{private_hist}[b, \text{bin}]$ ，再按 bin 合并块结果。
- 块内通常用 shared-memory atomic increments；块间合并可让一个输出工作项独占一个 bin。
- 基线 int64 输入/计数；可追加 uint8 输入布局对照，但必须单独报告字节数。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
zero(private_hist)
for item in block_tile: atomic_add(local[bin_id],1)
count[bin] = sum_b(private_hist[b,bin])
```

为什么是公认 GPU 任务

- GPU 适用性证据：CUDA 官方 histogram 示例展示并行直方图任务。
- 大量输入可并行处理；块私有化限制不同块争抢同一计数器。
- 适合评估原子、shared memory 和输入分布敏感性，不能只测均匀 bins。

适用规模 / 输入条件

全部输入落在同一 bin 时冲突会增多；bin 数太大时 shared-memory 私有化也受容量限制。

[HIST] NVIDIA CUDA histogram 示例

04 histogram：数据布局、OSP 与 Jac 要求

选择算法保持一致；最低可执行要求与高性能扩展分开。各项功能在后文词典解释。

D 设备数组 / view

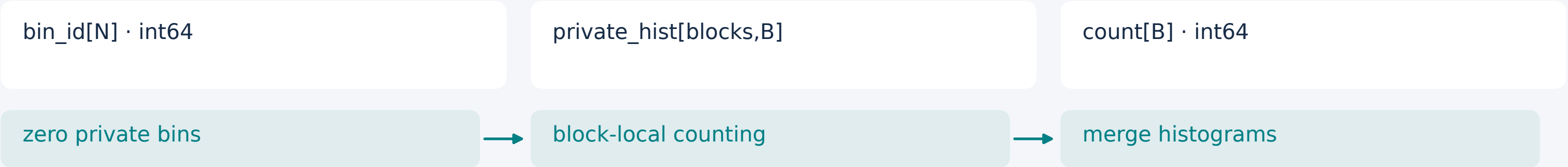
I 索引 / 控制流

P 任务 / 输出 / 原子

R 归约 / scan / 阶段

N 数值语义

物理表示（共享输入 / 独占或显式共享输出）



OSP 表达：独立 bin walker 可串行重复扫描，但不是所选私有化算法；常见算法需要块内共享计数。

最低 GPU 运行：Jac 必须提供

- D/I/P：整数数组、初始化、bin 下标范围与私有 histogram workspace。
- R/P：块内原子或等效局部计数；完成后按 bin 合并为全局 count。
- 阶段与容量管理：合并不能读取仍在更新的 private_hist。

高性能扩展 / 复用情况

- H：shared-memory histogram；warp 私有 bins / 分组更新减少热点竞争。
- 可根据 B/分布选择 tile；压缩输入 dtype 是独立布局优化。

当前状态

bool 条件与私有整数累计已有；跨线程计数、原子和 block scratch 缺失。

当前 GPU 后端能力依据：ptx_walker.jac / ptx_csr.jac；映射与扩展建议未实机验证。

04 histogram : benchmark 定义与正确性

以下为建议运行协议。微型例子由 Python CPU 数值模型复核；没有执行该 Jac/GPU benchmark。

输入规模、状态重置与指标

- $N=2^{20} / 2^{24}$, $B=16/256/4096$; int64 输入在主规模为128 MiB。
- 均匀、单一 bin、少数热点三类输入；按实际 $blocks \times B$ 报告 workspace。
- 初始化与合并都计入算法时间；每轮计数从零开始。

报告什么

items/s、总时间、不同分布下吞吐； $\text{sum}(\text{count})=N$ 必须成立。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
bins=[0,1,1,3], B=4
count=[1,2,0,1]
```

CPU 参考与关键检查

CPU bincount/整数计数；所有 bins 精确比较，检查 count 非负且总量守恒。

- 全同 bin、B 大小变化、尾 tile；保证无越界输入并验证计数没有丢失。
- 原子不丢更新与 int64 不溢出分别检查；不使用浮点计数。

GPU 对照 / 库路径

NVIDIA/CUB histogram 或等价 GPU 私有化实现；同 B、dtype、分布比较。

[HIST] NVIDIA CUDA histogram 示例

05 radix_sort : 整数 key / key-value 排序

选定算法：稳定 LSD radix sort , 4 bit 一轮

任务定义与算法步骤

对非负 32-bit key 升序排序，保留同 key 的原始次序和 payload 身份。

- 每轮取低位到高位 的 4-bit digit；计算 16 个 bucket 的位置并稳定 scatter。
- 共 8 轮覆盖 32 位；每轮 histogram/scan/scatter，数据在两个 buffer 间切换。
- 逻辑 key 范围 $0..2^{32}-1$ ，可先用 int64 列承载；仍需定义位宽与稳定性。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
for shift in [0,4,...,28]:
    digit = (key >> shift) & 15
    offsets = histogram_and_scan(digit)
    stable_scatter(key,payload,offsets); swap_buffers()
```

为什么是公认 GPU 任务

- GPU 适用性证据：NVIDIA CUB DeviceRadixSort 提供大数组整数/key-value 并行排序。
- digit 处理与散射可并行，通用排序是 GPU 数据处理中的常见原语。
- 选定 LSD 稳定多轮算法；不使用小数组 sorting network 来代表大规模排序。

适用规模 / 输入条件

8 轮搬运可能受带宽限制；短数组或极少量 key 不一定弥补 launch 成本。

[RADIX] NVIDIA CUB : DeviceRadixSort 源码

05 radix_sort：数据布局、OSP 与 Jac 要求

选择算法保持一致；最低可执行要求与高性能扩展分开。各项功能在后文词典解释。

- D 设备数组 / view
- I 索引 / 控制流
- P 任务 / 输出 / 原子
- R 归约 / scan / 阶段
- M 数学 / 位操作
- N 数值语义

物理表示（共享输入 / 独占或显式共享输出）



OSP 表达：每个 key 是逻辑记录；归属 bucket 每轮变化，显式构造重排图没有必要。

最低 GPU 运行：Jac 必须提供

- D/I/M：buffer、循环、shift/bitmask 或等价 digit 运算；不要求先有 uint32 类型。
- R/P：histogram/scan 与稳定的 bucket 内位置，scatter 目标唯一。
- N：固定 unsigned key 范围与 payload 稳定性；global atomic append 本身不保证稳定排序。

高性能扩展 / 复用情况

- H：block radix rank、局部 shared-memory exchange 与连续scatter。
- L：CUB SortPairs；库内部 digit 宽度可不同，语义必须相同。

当前状态

现有 int64/bool 不意味着支持 bitwise/shift、scan 或 scatter；这条路径需扩展。

当前 GPU 后端能力依据：ptx_walker.jac / ptx_csr.jac；映射与扩展建议未实机验证。

05 radix_sort : benchmark 定义与正确性

以下为建议运行协议。微型例子由 Python CPU 数值模型复核；没有执行该 Jac/GPU benchmark。

输入规模、状态重置与指标

- $N=2^{20} / 2^{24}$; int64 key/id 双缓冲主规模为 512 MiB , 另加排序 workspace。
- 均匀 32-bit、很多重复 key、已排序、逆序、0 与 $2^{32}-1$ 。
- 全部 8 轮纳入计时；输入每轮基准重置，避免只测已排序快路。

报告什么

keys/s、总排序时间、峰值显存；比较 key-only 与 key-value 时分别报告。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
keys=[3,1,3,0], ids=[0,1,2,3]
sorted=[0,1,3,3], ids=[3,1,0,2]
```

CPU 参考与关键检查

CPU stable sort 对 (key,id) 结果；检查排序单调、排列不丢失和同 key 的 id 顺序。

- 高位为 1 的 key 仍按 unsigned 范围排序；不要借有符号比较改变含义。
- 重复 key 稳定性和 payload 搬运正确；检查每轮 scatter 无目标冲突。

GPU 对照 / 库路径

CUB DeviceRadixSort SortPairs ; 明确 payload dtype、bit range 与 workspace。

[RADIX] NVIDIA CUB : DeviceRadixSort 源码

06 spmv : 稀疏矩阵 × 向量

选定算法：CSR 行并行 SpMV (可选 warp-per-row)

任务定义与算法步骤

$y[i] = \sum A[i,k] * x[k]$; A 为 CSR , x/y 稠密 ; 不是 matmul 的另一个接口。

- offsets 定义每行非零区间 ; indices[p] 找 x 的位置 , values [p]是权重。
- 每行独占 y[i] ; 行内串行或多个 lane 协作求和 , 工作 $O(\text{nnz})$ 。
- 不同于图 CSR , 数值 CSR 还需 values 列和 indirect gather x。

计算 / 数值示意 (不是 Jac/GPU 运行结果)

```
for row in parallel:
    y[row] = sum_p(values[p]*x[indices[p]])
    # p in offsets[row]:offsets[row+1]
```

为什么是公认 GPU 任务

- GPU 适用性证据 : cuSPARSE 提供 GPU SpMV , 是求解器/HPC 的标准稀疏基础算子。
- 不同稀疏行可并行 ; 长行可分给 warp , 工作量取决于 nnz 和行长分布。
- 主要考察 irregular gather、归约和负载均衡 ; 不要求构建逐乘法节点。

适用规模 / 输入条件

极短行、极不均匀行或差的 x 局部性会影响效率 ; 需按稀疏结构分组报告。

06 spmv：数据布局、OSP 与 Jac 要求

选择算法保持一致；最低可执行要求与高性能扩展分开。各项功能在后文词典解释。

D 设备数组 / view

I 索引 / 控制流

P 任务 / 输出 / 原子

N 数值语义

物理表示（共享输入 / 独占或显式共享输出）



OSP 表达：行可作为任务节点，非零关系可作为 edge；保留 OSP 需边权/query，直接 buffer 版可用 D/I。

最低 GPU 运行：Jac 必须提供

- D/I：CSR index/value buffers、变长循环与 x[indices[p]]间接读取。
- P：一行/行片段任务与 y 所有权；串行行内求和不要求 atomic。
- N：float64 求和容差；行内并行另需 R，不能照搬无权图邻接当成数值 CSR。

高性能扩展 / 复用情况

- H/R：warp-per-row、长行分段、行长分桶，减少分歧与不均衡。
- L：cuSPARSE SpMV；G：如果保留 Jac edge 权重/neighbor query 的OSP 接口。

当前状态

现有 CSR 能遍历节点；没有通用 values+indices+x gather。乘数项 chain 可算，但复制 x 是另一布局。

当前 GPU 后端能力依据：ptx_walker.jac / ptx_csr.jac；映射与扩展建议未实机验证。

06 spmv : benchmark 定义与正确性

以下为建议运行协议。微型例子由 Python CPU 数值模型复核；没有执行该 Jac/GPU benchmark。

输入规模、状态重置与指标

- $M=K=2^{20}$ ，平均 16 非零/行（约 16.8M nnz）；缩放 M/行长做 sweep。
- CSR 主数组约 $8(M+1)+16nnz+8K+8M$ 字节，不含库 workspace。
- 均匀短行、长尾行长、对角/带状和随机列；格式转换单独计时。

报告什么

nnz/s、约 2nnz 的算术吞吐、resident/端到端时间；记录稀疏结构。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
A=[[2,0,1],[0,3,0]], x=[1,2,3]
CSR offsets=[0,2,3], indices=[0,2,1]
y=[5,6]
```

CPU 参考与关键检查

CPU CSR/dense 小矩阵乘向量；浮点按每行参考和误差比较。

- 空行、长行、尾行、x 间接索引；输入定义为有效 CSR，重复项规则固定。
- 一线程一行与 warp 版本分别验证；转换时间与计算时间不混用。

GPU 对照 / 库路径

cuSPARSE SpMV；使用同 CSR/dtype/alpha=1/beta=0 与完整调用。

07 jacobi : 二维 stencil / Jacobi

选定算法：5-point Jacobi，双缓冲逐轮更新

任务定义与算法步骤

固定边界 U ；内部 $U_{\text{new}}[i,j] = (\text{上} + \text{下} + \text{左} + \text{右})/4$ ， T 轮。

- 每轮只读旧 U ，写新 U ；全轮完成后交换两个 buffer。
- 每个内部网格点同轮互不依赖；跨轮依赖明确，不能原地更新成 Gauss-Seidel。
- $O(T*HW)$ ；基准固定 T ，若改为收敛停止则额外需要全局残差归约。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
for t in [0,T):
    U_next[i,j] = .25*(U[i-1,j]+U[i+1,j]
                      +U[i,j-1]+U[i,j+1])
    preserve_boundary(); complete_stage(); swap()
```

为什么是公认 GPU 任务

- GPU 适用性证据：NVIDIA 提供 single/multi-GPU Jacobi solver 示例。
- 每轮大量规则网格点可并行，邻域访问与 stencil reuse 是常见 GPU/HPC 模式。
- 双缓冲消除同轮数据竞争；可进一步加载含 halo 的 tile。

适用规模 / 输入条件

小网格或很多极短迭代会受 launch 限制；本报告只分析单 GPU，未引入多 GPU 通信。

07 jacobi：数据布局、OSP 与 Jac 要求

选择算法保持一致；最低可执行要求与高性能扩展分开。各项功能在后文词典解释。

D 设备数组 / view

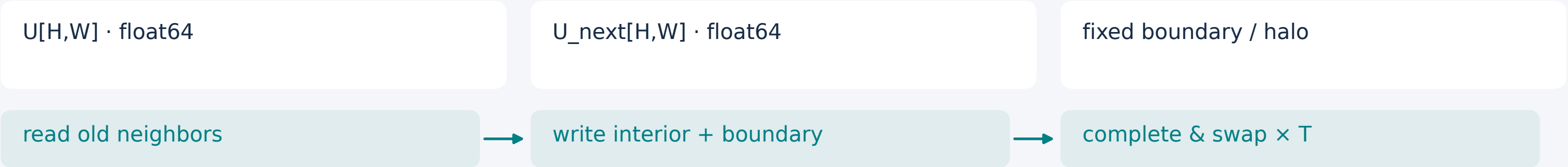
I 索引 / 控制流

P 任务 / 输出 / 原子

R 归约 / scan / 阶段

N 数值语义

物理表示（共享输入 / 独占或显式共享输出）



OSP 表达：逻辑格点/邻接可由 shape 隐式生成；不必由 CPU 创建 4HW 条实体边。

最低 GPU 运行：Jac 必须提供

- D/I/P：二维 view、邻域下标/边界 mask、唯一输出格点。
- R/运行时：每轮完成与 swap；两个 buffer 与边界生命周期。
- 固定 T 无需收敛归约；若加入误差停止，再增加 max/sum residual。

高性能扩展 / 复用情况

- H：含 halo 的 shared tile，复用邻域；边界线程也遵守 barrier 参与规则。
- 时间分块可减少 global 流量，但需正确处理跨步 halo 和边界。

当前状态

图只读和私有标量更新已有；共享数组写入与全轮 buffer 交换不是现有 walker 功能。

当前 GPU 后端能力依据：ptx_walker.jac / ptx_csr.jac；映射与扩展建议未实机验证。

07 jacobi : benchmark 定义与正确性

以下为建议运行协议。微型例子由 Python CPU 数值模型复核；没有执行该 Jac/GPU benchmark。

输入规模、状态重置与指标

- $H=W=1024/4096/8192$, $T=100$; 4096^2 的两个 float64 网格共 256 MiB。
- 边界固定，内部有限随机值；同时测矩形和非 tile 整数倍。
- 完整 100 轮计时；边界复制/保持与 swap 均纳入，结果最后回传。

报告什么

$GUpdates/s = T*(H-2)*(W-2)/seconds/1e9$; 同时报告总时间。

计算 / 数值示意 (不是 Jac/GPU 运行结果)

```
U=[[0,2,0],[4,0,6],[0,8,0]]
one step: interior U_next[1,1]=5
all boundary entries remain fixed
```

CPU 参考与关键检查

CPU 双缓冲 Jacobi，同样边界与 T；逐格比较，不用原地循环作参考。

- $T=1/2/100$ 、固定边界、尾 tile；输入/输出角色每轮交换正确。
- 若采用收敛停止，明确 norm、阈值、最大 T 并报告实际迭代数。

GPU 对照 / 库路径

NVIDIA single_gpu Jacobi 或同公式 CUDA stencil；与热传导/不同边界问题不能混比。

08 convolution : 图像卷积 / 平滑滤波

选定算法 : 可分离 Gaussian : 水平 + 垂直两遍

任务定义与算法步骤

核 $K(u,v)=h[u]*h[v]$; 默认半径 $R=3$ (7 taps) , 零填充边界。

- 先对每行做 1D 卷积到 temp , 再对每列做 1D 卷积到 out。
- 将 $(2R+1)^2$ 个乘加降为两遍各 $(2R+1)$ 个乘加 , 前提是核确实可分离。
- h 在 host 按 Gaussian 生成并归一化 ; GPU kernel 不必先支持 exp 来生成固定系数。

计算 / 数值示意 (不是 Jac/GPU 运行结果)

```
temp[i,j] = sum_u(h[u]*image[i,j+u])
out[i,j]  = sum_v(h[v]*temp[i+v,j])
# out-of-image samples are zero
```

为什么是公认 GPU 任务

- GPU 适用性证据 : 官方 convolutionSeparable sample 专门展示 GPU 可分离卷积。
- 大图像有独立输出像素与局部 reuse ; halo tile 便于协作读取。
- 选定两遍可分离滤波 , 不能把它当作任意 2D convolution 的通用实现。

适用规模 / 输入条件

半径很小或图像很小时搬运/launch 成本显著 ; 大 R 的算法选择可能变化。

[CONV] NVIDIA separable convolution 示例

08 convolution：数据布局、OSP 与 Jac 要求

选择算法保持一致；最低可执行要求与高性能扩展分开。各项功能在后文词典解释。

D 设备数组 / view

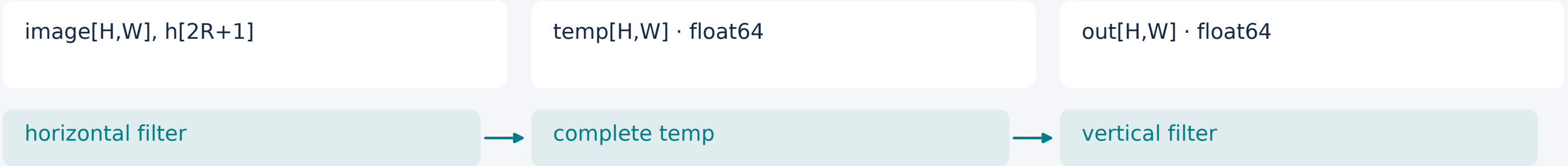
I 索引 / 控制流

P 任务 / 输出 / 原子

R 归约 / scan / 阶段

N 数值语义

物理表示（共享输入 / 独占或显式共享输出）



OSP 表达：像素邻域可用 view/隐式关系表达；每个输出像素任务拥有一个结果。

最低 GPU 运行：Jac 必须提供

- D/I/P：二维设备数组、R 循环、边界采样和独占输出。
- R：两遍 kernel 的依赖与 temp workspace；固定 h 可由CPU 准备。
- 只需常规乘加；不必为固定 filter 引入 device Gaussian 系数生成。

高性能扩展 / 复用情况

- H：横/竖方向各自的 halo tile、布局转置/pack，兼顾 coalescing。
- 系数缓存与小 R 专门化；L 可调用匹配可分离核的 GPU 图像算子。

当前状态

标量乘加可复用；邻域数组访问、temp 与多 kernel stage 仍需扩展。

当前 GPU 后端能力依据：ptx_walker.jac / ptx_csr.jac；映射与扩展建议未实机验证。

08 convolution : benchmark 定义与正确性

以下为建议运行协议。微型例子由 Python CPU 数值模型复核；没有执行该 Jac/GPU benchmark。

输入规模、状态重置与指标

- H=W=1024/4096/8192；R=3 主案例，另测 R=1/8；4096² 三网格共 384 MiB。
- sigma=1.5 的归一化 h；零填充，所有版本使用相同系数与padding。
- 两遍与 temp 读写计入完整时间；系数生成/上传单列。

报告什么

pixels/s、每图总时间；注明两遍 taps 数与中间空间，不与非分离核直接比。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
3x3 center impulse, R=1, h=[.25,.5,.25]
center output=.25; axial neighbors=.125
four corner outputs=.0625
```

CPU 参考与关键检查

CPU 直接 2D 外积核卷积作独立参考；与两遍结果按浮点容差比较。

- 边界/角落、非方图、奇数尺寸、不同 R；输出大小与零填充一致。
- 仅对可分离核使用该算法；物理转置的开销和临时空间不可隐藏。

GPU 对照 / 库路径

NVIDIA convolutionSeparable / 等价 GPU filter；固定相同 h、padding、dtype。

[CONV] NVIDIA separable convolution 示例

09 fft : 一维 complex FFT

选定算法 : radix-2 Cooley-Tukey DIT , bit-reversal 输入

任务定义与算法步骤

$N=2^m$, 计算前向 $X[k]=\sum x[n]*\exp(-2\pi i nk/N)$, 不做归一化。

- 输入先按 bit-reversal 重排 ; 每层执行不同跨度的蝶形 ($u+w*v, u-w*v$)。
- 共 $\log_2(N)$ 层、每层 $N/2$ 个独立蝶形 , 工作 $O(N \log N)$ 。
- complex128 可用实/虚两个 float64 列 ; twiddle 可在 CPU 预计算后上传。

计算 / 数值示意 (不是 Jac/GPU 运行结果)

```
z = bit_reverse_permute(x)
for length in [2,4,...,N]:
    (u,v) = butterfly_inputs(z,length)
    outputs = (u+w*v, u-w*v); complete_stage()
```

为什么是公认 GPU 任务

- GPU 适用性证据 : cuFFT 提供 GPU FFT ; 频谱、信号与科学计算广泛使用。
- 每层大量蝶形并行 ; 复杂数据布局与阶段依赖比标量 chain 更有代表性。
- 选择经典 radix-2 DIT ; cuFFT 的内部 radix/layout 可能不同。

适用规模 / 输入条件

各层 stride 不同 , 不能仅靠一种 SoA 顺序保证所有层合并访存 ; 小 FFT 常要批处理。

09 fft：数据布局、OSP 与 Jac 要求

选择算法保持一致；最低可执行要求与高性能扩展分开。各项功能在后文词典解释。

D 设备数组 / view

I 索引 / 控制流

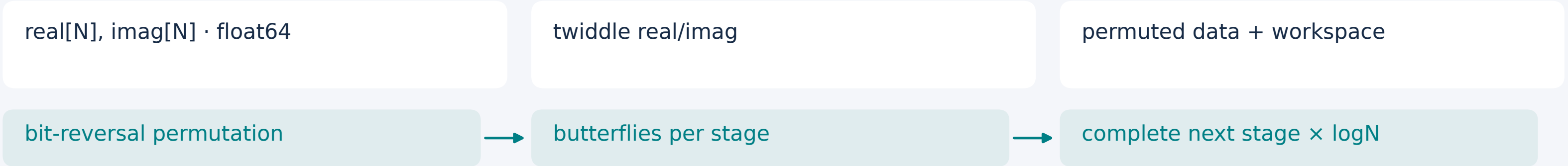
P 任务 / 输出 / 原子

R 归约 / scan / 阶段

M 数学 / 位操作

N 数值语义

物理表示（共享输入 / 独占或显式共享输出）



OSP 表达：逻辑蝶形是规则计算图，边可由索引公式生成；无需 CPU 实体化 $N \log N$ 节点。

最低 GPU 运行：Jac 必须提供

- D/I/P：动态跨度下标、每层唯一输出，bit-reversal permutation。
- M：复数乘加可展开为实/虚标量运算；位重排需 bit 运算或等效索引；无需新 complex 类型。
- R/N：跨层顺序、twiddle 生命周期、前向符号/归一化与 complex 容差。

高性能扩展 / 复用情况

- H：warp/block 内局部 FFT，减少跨层 kernel/global 流量；按 batch 与长度选 tile。
- L：cuFFT plan / workspace 与驻留 input ABI；库算法作为语义对照。

当前状态

fixed obj 的 re/im 标量可展开，但 FFT 所需数组、重排、蝶形层调度仍不支持。

当前 GPU 后端能力依据：ptx_walker.jac / ptx_csr.jac；映射与扩展建议未实机验证。

09 fft : benchmark 定义与正确性

以下为建议运行协议。微型例子由 Python CPU 数值模型复核；没有执行该 Jac/GPU benchmark。

输入规模、状态重置与指标

- $N=2^{10}/2^{16}/2^{20}$ ；主规模可用 batch=16；complex128 单个 2^{20} 数组 16 MiB。
- 正向 FFT，无归一化；有实/虚输入；总显存按 batch、双缓冲与 twiddle/workspace 计算。
- 计入 permutation+全部层；库 plan creation 单列，执行计时前完成预热。

报告什么

FFTs/s、总时间、batch 与长度；不把 plan 创建时间隐去后混称端到端时间。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
x=[0,1,0,0]
forward X=[1,-i,-1,+i]
```

CPU 参考与关键检查

小 N 用直接 DFT 作独立参考，大 N 用 NumPy/SciPy FFT；明确前向与 inverse 的 scaling。

- 单位脉冲、单频、全零、随机复数；real/imag 独立比较。
- bit-reversal、每层 butterfly 配对、批次 stride；同 dtype/ 方向对照。

GPU 对照 / 库路径

cuFFT complex128 forward；库内部算法不承诺等于选定 DIT

[FFT] NVIDIA cuFFT：GPU FFT 与计划

10 nbody : N-body 粒子作用力

选定算法：分块 all-pairs softened gravitational force

任务定义与算法步骤

每粒子输出加速度 $a_i = \sum G \cdot m_j \cdot (r_j - r_i) / (||r_j - r_i||^2 + \epsilon^2)^{3/2}$ 。

- 每个 target i 拥有三路累计；分块读取 source j，在多个 target 间复用位置/质量。
- 直接 all-pairs 工作 $O(N^2)$ ，不采用 Barnes-Hut； $\epsilon^2 > 0$ 避免奇点。
- 本 benchmark 只测一次 force evaluation；若加积分/时间步，应作为扩展单列。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
for target i in parallel:
    for source tile: for j in tile:
        delta = r[j]-r[i]; q = dot(delta,delta)+eps2
        a[i] += G*m[j]*delta/(q*sqrt(q))
```

为什么是公认 GPU 任务

- GPU 适用性证据：GPU Gems / NVIDIA 的直接 N-body 实现采用 tile 复用。
- 算术 $O(N^2)$ 而原始输入 $O(N)$ ，大量独立 target 和复用有利于 GPU。
- 示范多标量输出、动态数学函数、tile 协作，比复制每对粒子节点更直接。

适用规模 / 输入条件

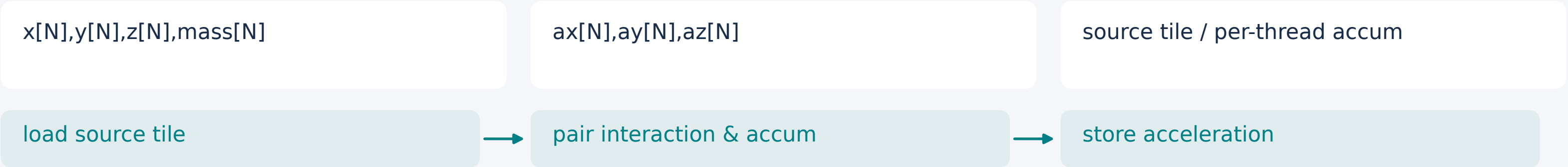
这是近距离软化的直接力模型； ϵ 、精度和 N 改变 workload，不把它和层次算法混比。

10 nbody：数据布局、OSP 与 Jac 要求

选择算法保持一致；最低可执行要求与高性能扩展分开。各项功能在后文词典解释。

- D 设备数组 / view
- I 索引 / 控制流
- P 任务 / 输出 / 原子
- M 数学 / 位操作
- N 数值语义

物理表示（共享输入 / 独占或显式共享输出）



OSP 表达：粒子可作为逻辑节点；all-pairs 关系隐式生成，避免 N^2 条实体 pair 边或节点。

最低 GPU 运行：Jac 必须提供

- D/I/P：共享 source 数组、两层索引、三个独占输出和有界累计状态。
- M：设备 sqrt/除法或匹配精度的 inverse-sqrt；不能在 CPU 预算全部 pair 来声称全 GPU。
- N：float64、固定 G/ϵ^2 与求和顺序；self 项处理要与参考一致。

高性能扩展 / 复用情况

- H：shared source tile 与 barrier，寄存器 target/三路累计，尽量复用每次加载。
- 流水线与数学近似只有在声明精度/误差后才启用。

当前状态

三路 float 私有状态可以表达累计；动态距离数学、数组和协作 tile 尚缺。

当前 GPU 后端能力依据：ptx_walker.jac / ptx_csr.jac；映射与扩展建议未实机验证。

10 nbody : benchmark 定义与正确性

以下为建议运行协议。微型例子由 Python CPU 数值模型复核；没有执行该 Jac/GPU benchmark。

输入规模、状态重置与指标

- $N=1024/8192/16384$; $G=1$, $\text{eps}^2=1\text{e}-4$, position/mass 固定随机有限值。
- 七个 float64 输入/输出列共 $56N$ 字节 , 另加 tile scratch ; 运算量按 N^2 记录。
- 一次 force kernel , 不混入渲染、积分或构建 N^2 pair 数据。

报告什么

interactions/s , $N^2/\text{seconds}$; 同时报告 dtype、 eps^2 与数学精度模式。

计算 / 数值示意 (不是 Jac/GPU 运行结果)

```
r0=(0,0,0), r1=(1,0,0), m0=m1=1
G=1, eps2=1: ax0=+1/(2*sqrt(2))
ax1=-1/(2*sqrt(2)); ay=az=0
```

CPU 参考与关键检查

CPU 直接 all-pairs float64 或更高精度小 N ; 逐分量比对加速度。

- 相同位置、零质量、少粒子/尾 tile、self contribution 与正 ϵ^2 。
- 量级差与误差 ; 不要求优化数学路径逐位等于单线程 CPU 累加。

GPU 对照 / 库路径

同模型 CUDA tiled N-body force ; 排除渲染/积分 , 参数与精度匹配。

11 kmeans : 聚类 / K-means

选定算法 : Lloyd : 最近中心分配 + 分组重算中心

任务定义与算法步骤

P 个 D 维点、K 个中心 ; 固定 T 次 Lloyd 迭代。

- assignment 计算平方距离并取最近中心 , 不需要 sqrt ; 同距离取较小中心 id。
- update 按 cluster 累加每维坐标与 count , 再计算均值。
- 空 cluster 保留旧中心 ; 固定初始中心与 T , 避免随机初始化差异。

计算 / 数值示意 (不是 Jac/GPU 运行结果)

```
label[p] = argmin_c(sum_d((x[p,d]-center[c,d])^2))
sums,counts = grouped_reduce(x,label)
center[c] = sums[c]/counts[c] if count>0 else old
complete_iteration(); repeat T
```

为什么是公认 GPU 任务

- GPU 适用性证据 : Rodinia 将 K-means 列为 CUDA/OpenCL 应用。
- 点的 assignment 高度独立 , 中心/维度可以分块复用 ; update 是分组归约。
- 更适合足够多点/中心/维度 , 展示 argmin、共享累计和迭代驻留。

适用规模 / 输入条件

很多点落到同一 cluster 时原子会竞争 ; K 很小/点数很少可能受阶段启动限制。

11 kmeans：数据布局、OSP 与 Jac 要求

选择算法保持一致；最低可执行要求与高性能扩展分开。各项功能在后文词典解释。

D 设备数组 / view

I 索引 / 控制流

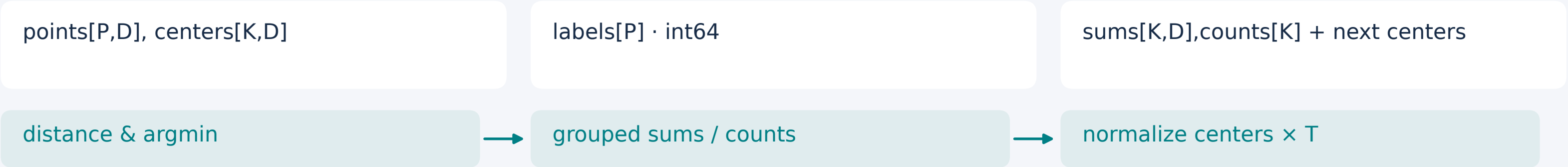
P 任务 / 输出 / 原子

R 归约 / scan / 阶段

M 数学 / 位操作

N 数值语义

物理表示（共享输入 / 独占或显式共享输出）



OSP 表达：点到中心关系隐式生成；避免 PK 条实体边。中心更新需共享归约，不能仅靠 walker 私有 total。

最低 GPU 运行：Jac 必须提供

- D/I/P：二维数组、distance/argmin 循环；int labels/counts 与 float 坐标混合。
- R：按 label 分组求和/count、阶段依赖和新旧 centers 生命周期。
- M/N：动态 count 的浮点除法、空 cluster 和 tie 规则、固定迭代 T

高性能扩展 / 复用情况

- H：shared centers、tile 距离、局部 partial sums 后合并，降低全局原子竞争。
- L 可用成熟 GPU 聚类实现；算法/初始化/空簇规则匹配才是有效对照。

当前状态

比较、bool 分支和标量差平方可复用；数组、argmin/grouped reduction 和共享更新缺失。

当前 GPU 后端能力依据：ptx_walker.jac / ptx_csr.jac；映射与扩展建议未实机验证。

11 kmeans : benchmark 定义与正确性

以下为建议运行协议。微型例子由 Python CPU 数值模型复核；没有执行该 Jac/GPU benchmark。

输入规模、状态重置与指标

- $P=2^{16}/2^{20}$, $D=32$, $K=32/128$, $T=10$; 主 points float64 约 256 MiB。
- 固定 seed 与初始中心；簇分布均衡和热点两类数据；附空簇案例。
- 全部 assignment+update+normalize 迭代计时；初始化/上传单列。

报告什么

每轮/总时间、距离计算数 $P*K*D$ ；记录 inertia 与实际迭代次数。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
points=[(0,0),(2,0),(10,0)]
initial centers=[(0,0),(10,0)]
labels=[0,0,1]; next centers=[(1,0),(10,0)]
```

CPU 参考与关键检查

CPU Lloyd 同中心、T、tie/空簇规则；比较 labels、counts、centers 与 inertia。

- tie、空簇、少点、尾 batch；浮点差异可能改变临界 label，要单独解释。
- 每轮完成后才使用新 centers；不能混入在线更新变成另一种算法。

GPU 对照 / 库路径

Rodinia/等价 GPU Lloyd；同初始化与停止条件，排除算法规则差异。

12 bfs : 图遍历 / 单源 BFS

选定算法 : level-synchronous frontier BFS (push)

任务定义与算法步骤

无权有向图 $G(V,E)$, 源 s ; 输出最短 hop distance , 未达为 -1 。

- 当前 frontier 的顶点并行展开出边 ; $\text{CAS}(\text{dist}[v], -1, \text{level}+1)$ 决定唯一首次发现。
- 成功发现者把 v 加到 next frontier ; 全层完成后再处理下一层。
- 典型工作 $O(V+E)$ (可达部分) ; 同顶点多条到达必须去重 , 不能无限重复排队。

计算 / 数值示意 (不是 Jac/GPU 运行结果)

```
frontier=[s]; dist[s]=0
for u in frontier in parallel: for v in out(u):
    if CAS(dist[v], -1, level+1)==-1: append(next,v)
complete_level(); swap_frontiers(); level+=1
```

为什么是公认 GPU 任务

- GPU 适用性证据 : Rodinia BFS 与 Gunrock frontier-based graph processing。
- 宽 frontier 可同时处理许多边 ; 需要图规模和可达结构提供并行度。
- 这是共享 dist/visited 的图算法 , 与每个独立 walker 完成私有 CSR FIFO 不同。

适用规模 / 输入条件

链图/窄 frontier 往往并行度不足 ; 高 degree hub 又可能负载失衡 , 需按图族报告。

12 bfs：数据布局、OSP 与 Jac 要求

选择算法保持一致；最低可执行要求与高性能扩展分开。各项功能在后文词典解释。

D 设备数组 / view

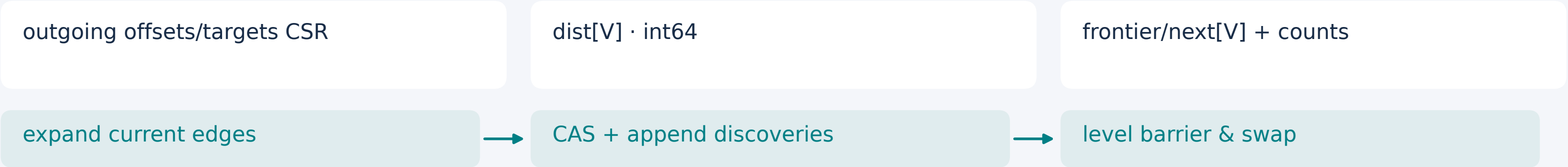
I 索引 / 控制流

P 任务 / 输出 / 原子

R 归约 / scan / 阶段

N 数值语义

物理表示（共享输入 / 独占或显式共享输出）



OSP 表达：可保留 node/edge/query 语义；需全局发现状态。bool 字段支持不等于 visited 的并发更新已支持。

最低 GPU 运行：Jac 必须提供

- D/I/P：CSR 读取、共享 dist、frontier buffers 与按活动顶点/边调度。
- R/P：CAS 首次发现、atomic append 或 scan compaction、容量和层边界。
- G 仅在保留 OSP query 时需要；CSR buffer 版也可通过 D/I 实现。

高性能扩展 / 复用情况

- H：warp/block 分摊长邻接行；edge-level work balancing，按 degree 分桶。
- 方向优化是另一算法扩展；本报告固定 push，不将其成本隐藏。

当前状态

已有只读 CSR 分支遍历与 bool 私有状态；共享 visited/dist、全局 frontier 与CAS 不支持。

当前 GPU 后端能力依据：ptx_walker.jac / ptx_csr.jac；映射与扩展建议未实机验证。

12 bfs : benchmark 定义与正确性

以下为建议运行协议。微型例子由 Python CPU 数值模型复核；没有执行该 Jac/GPU benchmark。

输入规模、状态重置与指标

- $V=2^{18}/2^{20}$, $E \approx 8V$; 随机宽图、网格、hub 图与链图分组。
- CSR+dist+两个 frontier 的 int64 主空间约 $8(E+4V+1)$ 字节。
- 多个固定源；包含孤立点与不连通图；初始化、每层控制与容量检测计入。

报告什么

总 BFS 时间、实际 examined edges/s ; TEPS 的边计数
定义必须写明。

计算 / 数值示意 (不是 Jac/GPU 运行结果)

```
edges: 0->1,0->2,1->3,2->3; vertex 4 isolated
source=0: dist=[0,1,1,2,-1]
```

CPU 参考与关键检查

CPU queue BFS ; dist 精确比较 ; 检查源、未达点与每条边的层次约束。

- 多路到达、环、自环、孤立点 ; next frontier 不重复且不越容量。
- 测试窄/宽 frontier ; 不能把重复 arrival 的私有 walker 报告当作标准 BFS distance。

GPU 对照 / 库路径

Rodinia / Gunrock push BFS ; 图方向、源、去重与计时范围一致。

13 pagerank : 图排序 / PageRank

选定算法 : pull power iteration + dangling mass correction

任务定义与算法步骤

有向图、阻尼 $d=.85$, 均匀 teleport ; 固定 $T=20$ 次迭代。

- 每个 v 读取 incoming neighbors , 累加 $\text{rank}[u]/\text{outdegree}[u]$, 唯一写 $\text{next_rank}[v]$ 。
- 出度为 0 的 rank 总质量经全局归约后均匀分配 , 避免质量丢失。
- 新旧 rank 双缓冲 ; 每轮读旧写新 , 不能混为异步原地 PageRank。

计算 / 数值示意 (不是 Jac/GPU 运行结果)

```
q = sum(rank[u] where outdegree[u]==0)
next[v] = (1-d)/V + d*(q/V +
                    sum_u_in_incoming(v)(rank[u]/outdegree[u]))
complete_iteration(); swap(rank,next)
```

为什么是公认 GPU 任务

- GPU 适用性证据 : Gunrock 文档提供 PageRank 与 GPU 图处理教程。
- pull 每个输出顶点唯一所有权 , 避免 rank 的 push 原子 scatter。
- incoming CSR/CSC 的行内 gather 和全局 q 归约共同提供大量并行工作。

适用规模 / 输入条件

degree 分布与入边访问局部性影响效率 ; 固定 T 与收敛停止是不同 benchmark 协议。

13 pagerank：数据布局、OSP 与 Jac 要求

选择算法保持一致；最低可执行要求与高性能扩展分开。各项功能在后文词典解释。

D 设备数组 / view

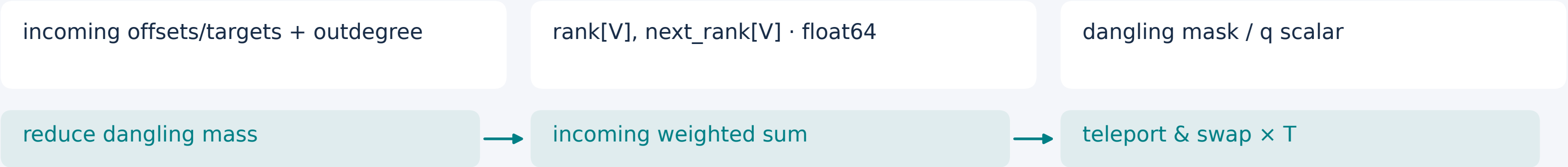
I 索引 / 控制流

P 任务 / 输出 / 原子

R 归约 / scan / 阶段

N 数值语义

物理表示（共享输入 / 独占或显式共享输出）



OSP 表达：task-root→只读 contributor leaf 可匹配现有 CSR 单轮求和结构；完整驻留迭代仍需新状态/阶段能力。

最低 GPU 运行：Jac 必须提供

- D/I/P：incoming CSR、远端 rank gather、每 v 独占 next_rank、双缓冲。
- R：dangling 全局求和与迭代依赖；inv_degree 可预计算，零出度仅进入 dangling。
- G 为保留 OSP handle/query 可选；GPU 数据驻留需明确更新与生命周期。

高性能扩展 / 复用情况

- H/R：warp-per-incoming-row、degree 分桶与 q reduction；保持 pull 的写入所有权。
- M 可选：动态除法；预计算 inv_degree 与 1/V 可全用乘加，但不能省略 dangling 修正。

当前状态

只读 CSR 邻居累计有可复用结构（未验证）；rank gather、全 GPU q/多轮驻留仍需扩展。

当前 GPU 后端能力依据：ptx_walker.jac / ptx_csr.jac；映射与扩展建议未实机验证。

13 pagerank : benchmark 定义与正确性

以下为建议运行协议。微型例子由 Python CPU 数值模型复核；没有执行该 Jac/GPU benchmark。

输入规模、状态重置与指标

- $V=2^{18}/2^{20}$, $E \approx 8V$; $d=.85$, $T=20$, 初始 $\text{rank}=1/V$ 。
- 入邻接可在 CPU 转置一次，转换成本单列；保留 outdegree 与 dangling 信息。
- 普通随机、hub 和包含 dangling 的图；完整全部迭代计时。

报告什么

每轮/总时间、processed edges/s、 $\text{sum}(\text{rank})$ 与 residual；固定 T 不报告收敛速度排名。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
2 vertices: edge 0->1; vertex 1 is dangling
start rank=[.5,.5], d=.85
next rank=[.2875,.7125]; total=1
```

CPU 参考与关键检查

CPU pull power iteration，同阻尼/teleport/T/dangling 规则；浮点逐顶点比较。

- dangling、孤立/环图、rank 总和约 1；测试新旧 rank 没有别名。
- 如改收敛停止，增加 residual reduction、阈值和实际迭代数报告。

GPU 对照 / 库路径

Gunrock/同 pull 语义 GPU PageRank；库的默认初始化/停止规则需适配。

14 alignment：序列全局比对 / 动态规划

选定算法：Needleman-Wunsch，anti-diagonal wavefront

任务定义与算法步骤

两序列长度 $L1/L2$ ； $match=+1$ ， $mismatch=-1$ ， $linear\ gap=-1$ ；输出 score。

- $DP[i,j]$ 取 $diagonal+substitution$ 、 $up+gap$ 、 $left+gap$ 三者最大。
- 同一 anti-diagonal 的单元相互独立；下一条对角线依赖已完成的前两条。
- 先计算得分，不做 traceback；若需 alignment path，另外增加方向存储/回溯。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
DP[i,0]=-i; DP[0,j]=-j
DP[i,j]=max(DP[i-1,j-1]+score(a[i],b[j]),
            DP[i-1,j]-1, DP[i,j-1]-1)
execute anti-diagonals in order
```

为什么是公认 GPU 任务

- GPU 适用性证据：Rodinia Needleman-Wunsch 为 GPU dynamic-programming benchmark。
- 长序列中部 anti-diagonal 较宽；多对序列可批处理以增加并行度。
- 规则 DP 与 wavefront 是另一类 GPU 任务，不能用无依赖逐元素map代替。

适用规模 / 输入条件

序列短、波前起止狭窄会限制并发；全 DP 存储 $O(L1L2)$ ，大型输入应先估算显存。

14 alignment：数据布局、OSP 与 Jac 要求

选择算法保持一致；最低可执行要求与高性能扩展分开。各项功能在后文词典解释。

D 设备数组 / view

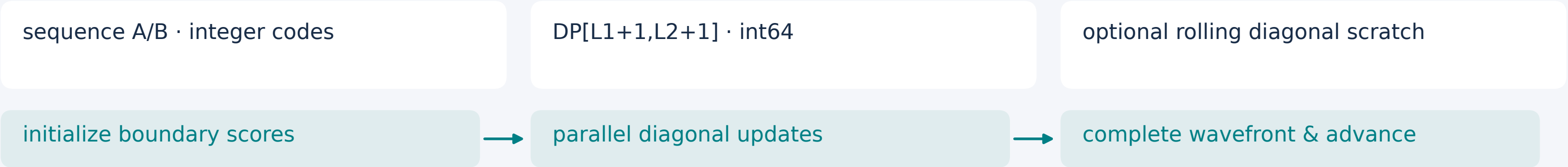
I 索引 / 控制流

P 任务 / 输出 / 原子

R 归约 / scan / 阶段

N 数值语义

物理表示（共享输入 / 独占或显式共享输出）



OSP 表达：DP 单元及依赖边可以隐式生成；全波前调度不是一个独立 walker 的简单 tail visit。

最低 GPU 运行：Jac 必须提供

- D/I/P：整数 DP 数组、max/比较、序列码读取与对角线坐标映射。
- R：anti-diagonal 阶段依赖；最小可一波前一 kernel，不需全 grid 内的 barrier。
- N：固定评分/linear gap、score 输出与 int64 范围；traceback 不在本基准范围。

高性能扩展 / 复用情况

- H：block/tile 波前，局部 scratch；滚动 diagonals 降空间，但同步 / 边界需正确。
- 批处理短序列改善并发；affine gap/Smith-Waterman 是不同算法，不能混入。

当前状态

整数比较/加法能复用；一般 DP 数组写入、max/helper 与波前多阶段尚缺。

当前 GPU 后端能力依据：ptx_walker.jac / ptx_csr.jac；映射与扩展建议未实机验证。

14 alignment : benchmark 定义与正确性

以下为建议运行协议。微型例子由 Python CPU 数值模型复核；没有执行该 Jac/GPU benchmark。

输入规模、状态重置与指标

- L1=L2=1024/4096 , batch=1/8 ; 4096² int64 DP 单对约128 MiB。
- 固定评分 +1/-1/-1 , 随机 A/C/G/T 和重复序列 ; score-only。
- 初始化与全部波前计时 ; 如果 rolling/tiled variant , 单列其 workspace。

报告什么

GCUPS = batch*L1*L2/seconds/1e9 ; 同时报告序列长、评分和batch。

计算 / 数值示意 (不是 Jac/GPU 运行结果)

```
A="AC", B="AG"
match=1, mismatch=-1, gap=-1
global alignment score=0
```

CPU 参考与关键检查

CPU row-major Needleman-Wunsch ; 整数 score 精确 , 微例全 DP 表可对照。

- 空序列、单字、全匹配、全不匹配、非方长 ; 边界不能漏初始化。
- 波前依赖与评分模式固定 ; 不要只验证最终 score 而掩盖错误中间状态。

GPU 对照 / 库路径

Rodinia Needleman-Wunsch / 同评分的 GPU wavefront 实现 ; 不直接套不同默认评分结果。

15 black_scholes：独立解析定价 / Black-Scholes

选定算法：欧洲 call/put 闭式公式 + normal CDF

任务定义与算法步骤

正 $S/K/T/\sigma$ 、无分红，计算同一模型的 call 与 put；每个 option 独立。

- $d1 = (\ln(S/K) + (r + \sigma^2/2)T) / (\sigma\sqrt{T})$ ， $d2 = d1 - \sigma\sqrt{T}$
- $\text{call} = S\Phi(d1) - K\exp(-rT)\Phi(d2)$ ；put 可用 put-call parity 求得。
- Φ 用 $.5*\text{erfc}(-x/\sqrt{2})$ 或声明误差的 CDF 近似；此处仅研究计算kernel。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
d1 = (log(S/K) + (r + .5*sigma*sigma)*T) / (sigma*sqrt(T))
d2 = d1 - sigma*sqrt(T)
call = S*Phi(d1) - K*exp(-r*T)*Phi(d2)
put = call - S + K*exp(-r*T)
```

为什么是公认 GPU 任务

- GPU 适用性证据：CUDA 官方 BlackScholes 示例为大量独立选项的计算任务。
- 每条记录相同公式，可并行执行并连续读取 SoA 输入；无跨 option 通信。
- 比 Triad 多出设备数学函数，但没有 frontier、共享原子或跨输出归约。

适用规模 / 输入条件

工作量太小时上传/launch 会占主导；模型参数和 CDF 近似必须固定才能比较。

[BLACK] NVIDIA CUDA Black-Scholes 示例

15 black_scholes：数据布局、OSP 与 Jac 要求

选择算法保持一致；最低可执行要求与高性能扩展分开。各项功能在后文词典解释。

D 设备数组 / view

I 索引 / 控制流

P 任务 / 输出 / 原子

M 数学 / 位操作

N 数值语义

物理表示（共享输入 / 独占或显式共享输出）



OSP 表达：一个 option 可是无边任务 node/walker；不用 host 构建金融实体关系图。

最低 GPU 运行：Jac 必须提供

- D/I/P：参数 SoA/view、唯一 call/put 输出、计算型 entry 与 batch。
- M：设备除法、log/sqrt/exp/erfc 或 CDF helper；普通 Python math 不能直接在 PTX 调用。
- N：float64/CDF 误差范围、正参数输入域；不需要先加入共享状态或归约。

高性能扩展 / 复用情况

- 数学函数 lowering / libdevice 与流水线；必要时明确 fast/strict 模式。
- 参数布局连续、两输出可同 kernel；库/近似算法要使用同一模型。

当前状态

标量 +/-/* 与 bool 已有；walker 设备除法/数学 helper 和参数数组仍缺。

当前 GPU 后端能力依据：ptx_walker.jac / ptx_csr.jac；映射与扩展建议未实机验证。

15 black_scholes : benchmark 定义与正确性

以下为建议运行协议。微型例子由 Python CPU 数值模型复核；没有执行该 Jac/GPU benchmark。

输入规模、状态重置与指标

- $N=2^{20}/2^{24}$, $r=.05$; $S、K\in[50,150]$, $T\in[.1,2]$, $\sigma\in[.1,.6]$ 。
- 四输入两输出 float64 主数组共 $48N$ 字节； $N=2^{24}$ 约768 MiB。
- 排除 $T=0/\sigma=0$ 的退化域，若加入必须定义对应极限分支。

报告什么

options/s、总时间、CDF 精度；比较必须是同模型和同 float dtype。

计算 / 数值示意（不是 Jac/GPU 运行结果）

$S=K=100, T=1, r=.05, \text{sigma}=.2$
 $\text{call}\approx 10.4505835722, \text{put}\approx 5.5735260223$

CPU 参考与关键检查

CPU double closed form , Phi 用 erfc；检查 put-call parity 与误差。

- 深价内/价外、短期限、小波动率；有限值、容差内非负价格与parity。
- CDF 近似误差与 GPU 浮点误差分别记录，不把近似模式混入精度对照。

GPU 对照 / 库路径

NVIDIA BlackScholes CUDA / 同模型 device kernel；source sample 精度可能不同，需适配。

[BLACK] NVIDIA CUDA Black-Scholes 示例

16 softmax：批量行归一化 / softmax

选定算法：stable max-subtract $\rightarrow$ exp/sum $\rightarrow$ normalize

任务定义与算法步骤

每行 $y_j = \exp(x_j - \max(x)) / \sum \exp(x_k - \max(x))$ ，有限输入且 $K > 0$ 。

- 先归约最大值，避免直接 exp 大输入溢出；再归约 exp 的和。
- 同一行输出彼此共享 max/sum；行与行独立，常用 warp/block-per-row。
- 基线可三阶段 buffer 实现；融合版把 exp/partial 留在片段内，减少中间流量。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
m[row] = max_j(x[row,j])
e[row,j] = exp(x[row,j]-m[row])
s[row] = sum_j(e[row,j])
y[row,j] = e[row,j]/s[row]
```

为什么是公认 GPU 任务

- GPU 适用性证据：cuDNN 提供 softmax 与避免溢出的 max-scaling 算法。
- 神经网络与批量概率变换常见；多行、多 lane 和 reduction 适合 GPU。
- 一个 row task 可分配给 warp/block；逻辑行节点不意味着物理大对象独立分配。

适用规模 / 输入条件

单个短行并发少；大 K 需要多段归约/资源规划。非有限输入策略需另行定义。

[SOFTMAX] NVIDIA cuDNN：稳定 softmax 算法

16 softmax：数据布局、OSP 与 Jac 要求

选择算法保持一致；最低可执行要求与高性能扩展分开。各项功能在后文词典解释。

D 设备数组 / view

I 索引 / 控制流

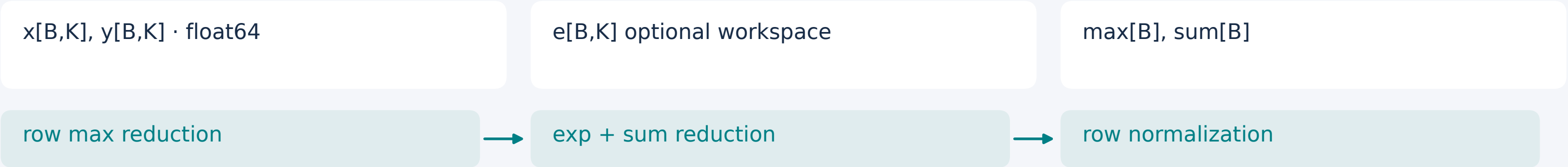
P 任务 / 输出 / 原子

R 归约 / scan / 阶段

M 数学 / 位操作

N 数值语义

物理表示（共享输入 / 独占或显式共享输出）



OSP 表达：行是自然逻辑任务，设备 view 为输入；多个线程协作时需有显式参与/归约契约。

最低 GPU 运行：Jac 必须提供

- D/I/P：行 view、动态列索引、输出 ownership 与 batch/row 映射。
- R/M：max 与 sum 归约、exp、除法；阶段完成与 m/s 的共享范围。
- N：固定 dtype 与 stable max-subtract 公式；不把 exp 的直算溢出当正确实现。

高性能扩展 / 复用情况

- H：warp/block row reduction，寄存器片段保存 exp，融合输出避免全局 e。
- L：匹配版本的 GPU DNN/tensor 库；接口可能换代，所需数值语义不变。

当前状态

scalar bool/compare 可作为组成部分；设备 exp、数组/行归约与协作都缺失。

当前 GPU 后端能力依据：ptx_walker.jac / ptx_csr.jac；映射与扩展建议未实机验证。

16 softmax : benchmark 定义与正确性

以下为建议运行协议。微型例子由 Python CPU 数值模型复核；没有执行该 Jac/GPU benchmark。

输入规模、状态重置与指标

- B=1024/16384 , K=128/1024/8192 ; B=16384,K=1024 时 x/y/e 共 384 MiB。
- 有限随机 logits 与 ± 1000 的压力例；基线非空行，输出与输入同形状。
- 完整 max+exp/sum+normalize；融合/三阶段实现分开记录 workspace 与流量。

报告什么

rows/s、elements/s、总时间和 max/sum 误差；注明行长度和 fused 状态。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
x=[1000,1000]
subtract max -> [0,0]
y=[.5,.5] (direct exp would overflow)
```

CPU 参考与关键检查

CPU float64 stable softmax；逐项比对，检查 $y \geq 0$ 、每行和 ≈ 1 与平移不变性。

- 单元素、长行、相等/极端有限 logits；tail row 与归约分组。
- max/subtract/sum 顺序和相同 dtype；如支持 mask，另定义全 mask 行语义。

GPU 对照 / 库路径

GPU DNN/tensor softmax；匹配 row axis、dtype、有限输入约定。

[SOFTMAX] NVIDIA cuDNN：稳定 softmax 算法

功能对照 1/2：每个 benchmark 的能力域

● 所选 buffer 算法所需；○ 常见优化 / 可选 OSP 或库路线。G 对保留图语言可选，D/I CSR 可先运行算法。

Benchmark	D	I	P	R	G	H	M	L	N
01 triad	●	●	●			○		○	●
02 reduction	●	●	●	●		○		○	●
03 scan	●	●	●	●		○		○	●
04 histogram	●	●	●	●		○		○	●
05 radix_sort	●	●	●	●		○	●	○	●
06 spmv	●	●	●	○	○	○		○	●
07 jacobi	●	●	●	●	○	○			●
08 convolution	●	●	●	●	○	○		○	●

当前标准 buffer 路径均需扩展；Triad 的单点表达式、串行 scalar fold / 部分 CSR gather 有可复用成分。

功能对照 2/2：每个 benchmark 的能力域

● 所选 buffer 算法所需；○ 常见优化 / 可选 OSP 或库路线。G 对保留图语言可选，D/I CSR 可先运行算法。

Benchmark	D	I	P	R	G	H	M	L	N
09 fft	●	●	●	●		○	●	○	●
10 nbody	●	●	●		○	○	●		●
11 kmeans	●	●	●	●	○	○	●	○	●
12 bfs	●	●	●	●	○	○		○	●
13 pagerank	●	●	●	●	○	○	○	○	●
14 alignment	●	●	●	●	○	○		○	●
15 black_scholes	●	●	●			○	●	○	●
16 softmax	●	●	●	●		○	●	○	●

当前标准 buffer 路径均需扩展；Triad 的单点表达式、串行 scalar fold / 部分 CSR gather 有可复用成分。

哪些任务最适合先推进当前 **system**

优先实现：少量能力即可覆盖多任务

- 先有 D/I/P：Triad → 一线程一行 SpMV → 简单 Jacobi / 可分离卷积；用 typed buffer 打通真正计算路径。
- 加入 R / 阶段：分层 sum、scan、DP/FFT 逐层 kernel；这组能力也是 frontier / grouped update 的基础。
- 加入 M：softmax、N-body、Black-Scholes；固定 filter/twiddle 先放 CPU 准备，避免不必要的 math API。

后续路线：按研究目标分支

- OSP 表达力：G 的 query/handle/edge payload；优先共享只读、独占输出的 pull/gather。
- 协作性能：H 的 tile/barrier/shuffle，优化 reduction、scan、卷积/FFT/N-body。
- 共享状态：CAS、append、原子与发布规则，支撑标准 BFS/histogram/K-means。
- L 为快速库对照/实用算子接入；不要把第三方 GPU 库成功等同于 Jac native backend 已支持。

优先级为工程建议；没有实际执行这些 benchmark 或给出性能排名。

OSP 的关键区分：CSR traversal 与标准 BFS

同样使用 CSR，也可能具有不同的状态、调度与终止语义。

现有 Jac CSR walker

- 共享图只读；每个 walker 有自己的有界 FIFO、私有标量状态和结果。
- arrival 触发 entry，重复到达仍执行；整个遍历由一个 thread 完成。
- 适合独立读取/累计；可处理分支和重复 arrival，与全局 frontier 调度不同。

所选 GPU BFS

- 整个 frontier 由许多线程协作扩展；dist/visited 是跨线程共享状态。
- CAS 对每个 v 决定唯一首次发现；next frontier 要去重、容量与层边界。
- 需要 P/R 的全局状态/队列契约；bool 属性和 CSR 上传是组成部分，不能自动补齐这些语义。

一个更接近现有 **OSP** 的切入点：**pull gather**

以下为未验证的表示建议，仅说明哪些结构可能复用；不是新 benchmark 已经成功运行的结论。



每个 task-root 连接 incoming contributor leaf；叶子只读，可共享；不让 leaf 再沿原图无限遍历。

现有 CSR + 单类型 float/bool role schema 可表达类似只读求和结构。

完整 PageRank 仍要 dangling mass、rank 更新、双缓冲与多轮驻留；标准数组算法需 D/I/P/R。

建议依据：当前 typed tail visit / 只读 CSR；选定 PageRank 算法来源 Gunrock。

实现后的报告格式：数据、阶段与 **correctness** 需一起出现

建议后续运行的 JSON/表字段。这里展示 schema，不是测量结果。

字段组	具体记录内容
标识	benchmark、算法变体、输入 hash/seed、规模/分布、dtype、参数/迭代数
环境	GPU/架构、驱动/编译器/库版本、Jac source SHA、math/precision mode
准备成本	CPU 构图/格式转换/初始化、plan/JIT、H2D、workspace/峰值显存
时间	resident kernel、全部 stage、D2H、端到端；warmup/repeats 与 median/p90
工作量	实际 nnz/edges/updates/iterations、有效字节数定义与吞吐指标
正确性	reference、容差、max abs/rel error、exact integer / stable payload / graph checks

D：设备 **buffer/view**、驻留与 **workspace**

功能词典：这些是 GPU 编译器/运行时的能力需求。很多语法在 CPU Jac 已存在，但 GPU 尚无对应 lowering。

功能定义与具体用途

- **buffer** 是有类型/长度的设备存储；**view** 额外描述 offset/shape/stride，多个任务共享底层。
- CSR 的 index/value 列、FFT 的 re/im 列、DP/图像二维网格都属于该能力，而不是 Python 对象指针。
- 驻留与分配复用不同：当前 runtime 复用容量但重打包/上传；workspace 用于 partial/temp/frontier。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
addr(i,j) = base + itemsize*(offset+i*s0+j*s1)
input: shared read-only; output: explicit ownership
workspace allocated on host, reused after completion
```

Jac 需要哪一层的支持

- 类型/编译器：设备句柄、dtype/stride 参数、load/store 与 alias/边界规则。
- 运行时：分配/释放、跨调用驻留、输入更新/失效与各 stage 的临时空间生命周期。
- 最低不需要 GPU malloc 或完整对象堆；固定 buffer 与 host 分配可以先覆盖所有任务。

当前状态

已有 scalar SoA；通用 arrays/views、tensor outputs 与数据驻留契约尚缺。

I：一般循环、动态索引与 **helper lowering**

功能词典：这些是 GPU 编译器/运行时的能力需求。很多语法在 CPU Jac 已存在，但 GPU 尚无对应 lowering。

功能定义与具体用途

- 用户 for/while 不同于运行时内部的 traversal loop；FFT stage、CSR p 循环、DP 索引都要编译。
- gather 读取 `x[index]`，scatter 写 `out[index]`；前者要有效索引，后者还需目标 ownership。
- max/min、len/range、static helper 可展开/内联；普通 Python generator/method 不能直接调用于 PTX。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
for p in offsets[row]:offsets[row+1]:
    total += values[p]*x[indices[p]]
row = task_id // width; col = task_id % width
```

Jac 需要哪一层的支持

- 编译器：局部变量、循环控制流/SSA、整数寻址与混合 int/float kernel 数据。
- 整数整除/取模、max 与边界 mask 可按任务需要增加；当前 walker 算术集合很有限。
- CPU 可以保留 shape 检查、格式/固定系数准备；数据依赖的计算主体需真正在 GPU。

当前状态

GPU walker 支持 self 字段赋值/if，但一般循环、局部变量与 helper 尚缺。

P：任务所有权、CAS 与共享状态

功能词典：这些是 GPU 编译器/运行时的能力需求。很多语法在 CPU Jac 已存在，但 GPU 尚无对应 lowering。

功能定义与具体用途

- 唯一 writer 的 output 可以直接 store：Triad 每 i、SpMV 每 row、PageRank pull 每 v。
- BFS 的不同父节点可能同时发现 v；CAS 的 winner 才能写一次新 distance 并排入 next。
- histogram/K-means 共享累计则需要 atomic 或分组 partial；bool 支持不等于共享 visited 并发安全。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
unique output: out[task_id] = value
first discovery: if CAS(dist[v],-1,new)==-1: append(v)
shared count: atomic_add(bin/count, delta)
```

Jac 需要哪一层的支持

- 语言/IR：私有、共享只读、共享可写、独占输出与操作原子性分别定义。
- 编译器：task/grid 映射、atomic/CAS lowering；运行时：frontier 容量、初始化和完成后发布。
- 坐标可由 lane 生成；compute entry 可无需 visit，或用空 typed tail visit 适配现有控制形式。

当前状态

现有只有私有 state 与只读图，通用共享更新/输出 buffer 不支持。

R : reduce、scan、compaction 与全局阶段

功能词典：这些是 GPU 编译器/运行时的能力需求。很多语法在 CPU Jac 已存在，但 GPU 尚无对应 lowering。

功能定义与具体用途

- reduce 把序列合成一个 sum/max；scan 为每项保留前缀；compaction 用 prefix 位置打包保留项。
- grouped reduce 按 cluster/bin/key 合并；wavefront/FFT/Jacobi 需要按依赖完成整个阶段。
- block barrier 不能当全 grid 阶段边界；先用 host 有序提交多个 kernel 即可实现层次算法。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
flags -> exclusive_scan -> positions -> compact
tile partials -> reduce -> scalar
stage A finishes -> stage B consumes its buffer
```

Jac 需要哪一层的支持

- 编译器：操作/单位元/axis/分组与写入唯一位置；归约树/顺序也要明确。
- 运行时：stream 完成/事件依赖、workspace 与 swap、错误/容量状态；不要求 device kernel launch。
- BFS 可用 atomic append 而不必先有 scan；row-serial SpMV 也不必先有跨 lane reduce。

当前状态

私有 scalar fold 已有；通用集合算子与多 kernel pipeline 尚缺。

G：图 handle/query、edge payload 与隐式关系

功能词典：这些是 GPU 编译器/运行时的能力需求。很多语法在 CPU Jac 已存在，但 GPU 尚无对应 lowering。

功能定义与具体用途

- handle 是设备可解析的身份（如类型+索引）；query 读取邻居/边，visit 则安排下一次 entry。
- BFS 是 outgoing CSR，PageRank pull 是 incoming CSR；SpMV 的 edge 权重与图 targets 不同。
- stencil/DP/N-body 的规则关系可由索引隐式生成；无需实体化 HW 或 N^2 条边。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
handle -> type/row -> field columns
query(node,relation) -> iterator/edge payload
implicit neighbor = coordinate_function(shape,position)
```

Jac 需要哪一层的支持

- 保留 OSP：类型化 handle、query iterator、receiver、edge SoA、多 schema 与引用 remap。
- 普通 buffer 路线可以直接使用 D/I CSR，不强制先实现整个 OSP 查询系统。
- 若用隐式逻辑 node/edge，要定义身份、边界/终止与布局；不是已经存在的新 Jac 语法。

当前状态

当前 typed tail visit 与 CSR FIFO 可分支；普通 query/edge payload 与任意 handles 缺失。

H : warp/block、shared memory 与布局协作

功能词典：这些是 GPU 编译器/运行时的能力需求。很多语法在 CPU Jac 已存在，但 GPU 尚无对应 lowering。

功能定义与具体用途

- 现有一个 walker 是一个 thread；协作任务需要把输出片段分给多个参与线程。
- shared 是 tile/cache/局部计数的地址空间，barrier 保证参与者按正确顺序使用；寄存器状态仍私有。
- coalescing 看同条 load 的 lane 地址；shared bank conflict 与 register spill 是其他性能问题。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
cooperative_load(tile); synchronize_participants()  
each thread computes owned fragment  
synchronize_before_overwriting_shared_tile()
```

Jac 需要哪一层的支持

- 语言/编译器：协作范围、参与掩码、scratch 地址空间、barrier 和 per-thread fragment。
- 后端/运行时：资源与 launch 配置、layout/pack、尾部线程的同步规则。
- H 多为性能扩展；正确的逐层 buffer kernel 可以先不做协作，但不能声称性能已经相同。

当前状态

已有 private scalar 与 global SoA；未有显式 warp/block 协作的用户语义。

M1：设备除法、**sqrt / exp / log / erfc**

功能词典：这些是 GPU 编译器/运行时的能力需求。很多语法在 CPU Jac 已存在，但 GPU 尚无对应 lowering。

功能定义与具体用途

- N-body 的动态距离、Black-Scholes 的 log/CDF、softmax 的 exp 与归一化需要设备数学。
- GPU walker 当前只接受有限 scalar 运算；CPU import math 并不会生成设备 callable。
- 固定 filter 系数与 FFT twiddle 可 host 预计算；不能预计算所有数据依赖 pair/option 来替代 GPU 算法。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
softmax: e = exp(x-row_max); y=e/row_sum
nbody: 1/(q*sqrt(q))
Phi(z) = .5*erfc(-z/sqrt(2))
```

Jac 需要哪一层的支持

- 编译器/链接：数学 intrinsic/device function 或 libdevice；类型/精度签名明确。
- 后端：domain/特殊值行为、strict 与近似数学选项；除法不是现有 walker 的默认操作。
- 库路线可让对应 GPU kernel 完成；只在 CPU 调 math 不满足计算主体上 GPU。

当前状态

动态数学函数、一般 helper 与 walker 除法尚未支持。

[MATH] NVIDIA libdevice：设备数学函数

M2：复数、bitwise / shift 与数据重排

功能词典：这些是 GPU 编译器/运行时的能力需求。很多语法在 CPU Jac 已存在，但 GPU 尚无对应 lowering。

功能定义与具体用途

- FFT 复数可用 re/im 两列；乘法展开为四个实乘与加减，不强制先加入 native complex 类型。
- radix 的 $\text{digit}=(\text{key} \gg \text{shift}) \& \text{mask}$ 和 FFT bit-reversal 需要位操作或等价整数索引算法。
- 支持 bool 的 and/or/not 与支持整数 bitwise $\&/\gg$ 是不同类型/操作；不能互相推断。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
(ar+i*ai)*(br+i*bi)=(ar*br-ai*bi)+i*(ar*bi+ai*br)
digit=(key >> shift) & 15
bit_reverse_index(i, logN) -> permuted address
```

Jac 需要哪一层的支持

- 类型/编译器：位宽、逻辑 shift、mask/整数索引、复数数组/字段 load-store。
- 数据 ABI：key 可 int64 承载 unsigned32 范围；complex128 可展开为两 float64，无需混 dtype。
- 重排要保持输入/输出 alias 与 scatter 唯一性；radix 稳定 rank 需另有 R/P 规则。

当前状态

已有 bool 逻辑和 scalar 实数运算；bitwise/shift 与一般数组重排仍缺。

[RADIX] NVIDIA CUB : DeviceRadixSort 源码

L：库 FFI、compiled wrapper 与 GPU tensor ABI

功能词典：这些是 GPU 编译器/运行时的能力需求。很多语法在 CPU Jac 已存在，但 GPU 尚无对应 lowering。

功能定义与具体用途

- Jac host 发起 GPU 库调用；库在设备计算。这与 Jac 自己 lowering 计算主体是两条路线。
- cuFFT/cuSPARSE 等需 device pointer、dtype、stride/descriptor、plan/handle、workspace 和 stream。
- CUB 主要是 C++ 模板：通常要 compiled wrapper 或 intrinsic 接入，不能假设 ctypes 直接调用模板。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
host Jac -> GPU library wrapper -> device kernel
describe buffers / plan / temporary storage
wait/check completion -> publish/read outputs
```

Jac 需要哪一层的支持

- 运行时：ABI、库探测、handle/context/stream 生命周期，异步完成和错误传播。
- shape/dtype/layout 适配；baseline 不能悄悄 CPU fallback，完整调用/plan/pack 时间分别报告。
- 库内部算法可不同；比较相同结果语义和输入域，不声称它实现了本文同一代码结构。

当前状态

已有 ctypes CUDA Driver API；各 GPU tensor/library wrapper 未统一接入。

N：误差、稳定性、dtype 与确定性

功能词典：这些是 GPU 编译器/运行时的能力需求。很多语法在 CPU Jac 已存在，但 GPU 尚无对应 lowering。

功能定义与具体用途

- 整数 scan/count/distance/DP 应精确；float64 tree sum、PageRank、FFT 与 K-means 各有误差契约。
- stable radix 是保持同 key 的记录顺序；stable softmax 是避免溢出的数值公式，两者含义不同。
- FMA/数学近似/归约树会改变舍入；固定树可稳定复现，但未必逐位等于 CPU 左折叠。

计算 / 数值示意（不是 Jac/GPU 运行结果）

```
exact integers != tolerance-based floats
deterministic grouping != CPU-bitwise identity
FP32 performance must compare with FP32 baseline
```

Jac 需要哪一层的支持

- 语言/编译器：dtype、求和顺序/fast math、tie/空簇/边界/评分与特殊输入规则。
- 验证：数学参考误差、重复运行稳定性、payload/图身份与精确整数分别检查。
- 性能报告：同算法语义/精度/范围；不能把少阶段、少迭代或低精度快路混进成绩。

当前状态

scalar 类型/错误与 bool 已有；新 tensor/atomic/math 路径需重新定义并验证契约。

[FLOAT] NVIDIA：浮点舍入与 FMA

参考资料 1/4：官方来源与算法依据

访问日期 2026-10-02；算法解释为本报告自述。API/示例路径和可用性以目标安装版本为准。

STREAM	BabelStream：GPU 内存带宽任务 github.com/UoB-HPC/BabelStream
TRIAD	BabelStream CUDA Triad 源码 github.com/UoB-HPC/BabelStream/blob/main/src/cuda/CUDASStream.cu
REDUCE	NVIDIA CUB：device-wide reduction nvidia.github.io/cccl/unstable/cub/api/structcub_1_1DeviceReduce.html
SCAN	GPU Gems 3 Ch.39：工作高效 scan developer.nvidia.com/gpugems/gpugems3/part-vi-gpu-computing/chapter-39-parallel-prefix-sum-scan-cuda
CUBSCAN	NVIDIA CUB：DeviceScan nvidia.github.io/cccl/unstable/cub/api/structcub_1_1DeviceScan.html
HIST	NVIDIA CUDA histogram 示例 github.com/NVIDIA/cuda-samples/tree/master/cpp/2_Concepts_and_Techniques/histogram

参考资料 2/4：官方来源与算法依据

访问日期 2026-10-02；算法解释为本报告自述。API/示例路径和可用性以目标安装版本为准。

RADIX	NVIDIA CUB : DeviceRadixSort 源码 github.com/NVIDIA/cccl/blob/main/cub/cub/device/device_radix_sort.cuh
SPARSE	NVIDIA cuSPARSE : SpMV / CSR docs.nvidia.com/cuda/cusparses/index.html
JACOBI	NVIDIA Jacobi solver / single_gpu github.com/NVIDIA/multi-gpu-programming-models
CONV	NVIDIA separable convolution 示例 github.com/NVIDIA/cuda-samples/tree/master/cpp/2_Concepts_and_Techniques/convolutionSeparable
FFT	NVIDIA cuFFT : GPU FFT 与计划 docs.nvidia.com/cuda/cufft/index.html
NBODY	GPU Gems 3 Ch.31：直接 N-body developer.nvidia.com/gpugems/gpugems3/part-v-physics-simulation/chapter-31-fast-n-body-simulation-cuda

参考资料 3/4：官方来源与算法依据

访问日期 2026-10-02；算法解释为本报告自述。API/示例路径和可用性以目标安装版本为准。

RODINIA	UVA Rodinia：GPU 应用 benchmark suite rodinia.cs.virginia.edu/
GUNROCK	Gunrock：GPU graph / frontier abstraction gunrock.github.io/gunrock/
PR	Gunrock：PageRank 算法教程 gunrock.github.io/gunrock/gunrock.wiki/PageRank.html
BLACK	NVIDIA CUDA Black-Scholes 示例 github.com/NVIDIA/cuda-samples/tree/master/cpp/5_Domain_Specific/BlackScholes
SOFTMAX	NVIDIA cuDNN：稳定 softmax 算法 docs.nvidia.com/deeplearning/cudnn/backend/latest/api/cudnn-ops-library.html#cudnnsoftmaxalgorithm-t
CUDA	NVIDIA CUDA Best Practices：访存与同步 docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html

参考资料 4/4：官方来源与算法依据

访问日期 2026-10-02；算法解释为本报告自述。API/示例路径和可用性以目标安装版本为准。

MATH

NVIDIA libdevice：设备数学函数

docs.nvidia.com/cuda/libdevice-users-guide/index.html

FLOAT

NVIDIA：浮点舍入与 FMA

docs.nvidia.com/cuda/floating-point/index.html

源码能力依据与验证范围

独立 benchmark PDF ; 原 matmul PDF 和算法/后端源码保持原样。

[jac/jaclang/compiler/backends/native/ptx_walker.jac](#)

SHA256: 4db7c15f69c117afd12cf4b432e4d0e45c1bfd737d02f06f98197315ef0f993f

[jac/jaclang/compiler/backends/native/ptx_csr.jac](#)

SHA256: 3a90d599a413445cabeaddc64924cc7469ee0838e59972d39781006a38fbd21d

[jac/jaclang/runtime/gpu.jac](#)

SHA256: bfeb83df77115bebb17a523b7ce8284e17beac676cb69101db5122aed2a457e4

[jac/jaclang/runtime/gpu_memory.jac](#)

SHA256: 5f86b9bf57f72f4c085b0f8e717f54a0eb22da89b241d3ae1e67bcecf6f36929f

[scripts/gpu_walker/runtime.md](#)

SHA256: b397839620dde100af7f006d797c54d9321390bad09095508595b6bb3d528b8d

已复核 16 个微型数值例子（Python CPU）：整数/序列/图规则及 float64 数值。

没有执行这些 Jac/GPU benchmark , 没有证明其运行、性能或 GPU 正确性。功能支持结论是源码分析。